

Term Paper Report - Group 1

Reference Research Paper: LeaFTL: A Learning-Based Flash Translation Layer for Solid-State Drives

CS531-Memory Systems and Architecture

1st Aman Saini

dept. of Electrical Engineering
Indian Institute of Technology, Ropar
Rupnagar, India
2020eeb1155@iitrpr.ac.in

2nd Ishaan Sethi

dept. of Electrical Engineering
Indian Institute of Technology, Ropar
Rupnagar, India
2020eeb1174@iitrpr.ac.in

Abstract—Solid State Drives (SSDs) store data in NAND flash memory. However, these NAND flash cells do not support direct writes; the present data must be erased to write new data. Therefore, out-of-place data updates are performed. Because of these out-of-place updates, a mapping table is maintained which maps the Logical Page Address (LPA) to the Physical Page Address (PPA). The Flash Translation Layer (FTL) maintains this mapping along with some other core functions like Garbage Collection (GC), Wear Leveling, etc. The mapping table is stored in Flash memory and cached in the on-device DRAM cache. Modern SSDs implement page-level mapping table, which has a huge memory footprint. LeaFTL [1] aims to reduce this size by approximating LPA-PPA mappings through linear segments, wherever possible. This term paper project aims to improve on the base-LeaFTL research paper to reduce the mapping table size even further.

I. INTRODUCTION

FTL is a core component of SSD, which maintains functions like GC, Wear Leveling, and Mapping table. The mapping table is cached in on-device DRAM and generally stores page-level mappings. The SSD receives writes and reads in the form of a starting address and a write/read length. The average write length spans multiple pages, depending upon a specific load. LeaFTL exploits this fact and places contiguous logical pages in contiguous physical pages. The mappings for these pages can then be approximated using a single segment. It implements a write buffer that stores the pages to be written in flash memory. When the write buffer size becomes equal to a block size, a free block is allocated, and the accumulated writes are flushed to the flash memory. The pages in the allocated block are written in order of increasing LPAs. Then a piece-wise linear regression is used to form the segments from the new LPA to PPA mappings.

II. BASE PAPER DETAILS

A. Inaccurate and Accurate Segments

LeaFTL has the ability to learn two types of segments: (1) Accurate and (2) Inaccurate. Accurate segments correctly map an LPA to its corresponding PPA. Inaccurate segments

are associated with an error margin γ . It is ensured that the accurate PPA lies within $\pm\gamma$ of the PPA predicted by the LeaFTL mapping table.

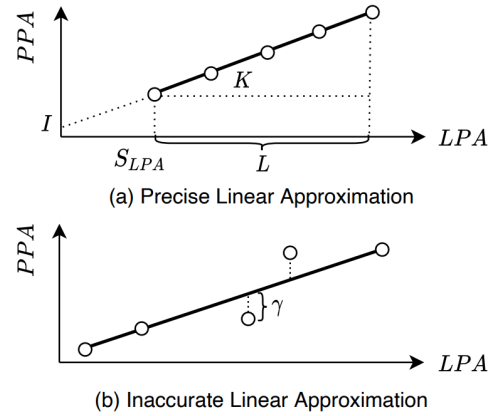


Fig. 1. Visualization of learned index segments. The PPA corresponding to an LPA can be obtained as: $PPA = f(LPA) = K * LPA + I$, where $LPA \in [S_{LPA}, S_{LPA} + L]$. Here K is slope and I is intercept, [1]

The learned Index Segments are represented using 8 Bytes. 1 Byte is reserved for S_{LPA} , 1 Byte of length L , 2 Bytes for slope K (stored as a 16-bit floating point) and 4 Bytes for intercept I , which represents a PPA. So, even if a LeaFTL mapping represents a single LPA to PPA mapping, size does not exceed that of a Page-Level Mapping table. Here S_{LPA} , which should ideally be 4 Bytes, is reduced to 1 Bytes, as LeaFTL groups 256 contiguous Logical Pages into Logical Frames. Each segment then maps the LPAs that belong to a single frame. Therefore, within a frame, only 1 Byte is required to address a logical page. Figure 2 shows the space distribution along with accurate and inaccurate segments.

B. Log Table

LeaFTL stores these segments for each frame separately in the form of a log table. The latest LPA to PPA mapping

	1B	1B	2B	4B
Segment	S_{LPA}	L	K	I

Type	LPAs	PPAs	Index Segment
Accurate	[0, 1, 2, 3]	[32, 33, 34, 35]	0 3 1.00 32
Approximate	[0, 1, 4, 5]	[64, 65, 66, 67]	0 5 0.56 64

Fig. 2. Mapping entry storage space distribution, along with accurate and inaccurate segment examples, [1]

segments are placed on top level, and the bottom levels store past LPA to PPA mapping segments. In the look-up operation, the mapping is searched from top to bottom of the log-table. Periodic compaction of the log table is carried out that reduces the number of levels and the size of the log table.

The Log-table is a major component of LeaFTL and highly optimized for its requirements. The LeaFTL mapping segments usually map contiguous logic page addresses, and the log table functions are highly tailored to optimally process such segments.

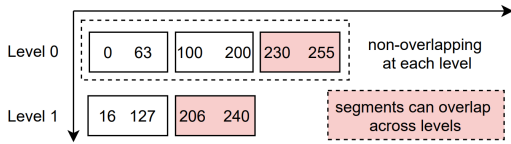


Fig. 3. Log structured table to store the learned segments in temporal order. LeaFTL ensures no overlapping segment is present on the same level, [1]

III. IMPLEMENTATION AND RESULTS

The authors of the LeaFTL paper implemented this mapping scheme in WiscSim, which is a python based simulator [Link](#). The LeaFTL functionality was replicated by porting the WiscSim source code to SimpleSSD, which is an advanced full-system Solid-State Drive Simulator [Documentation](#). LeaFTL was tested in SimpleSSD for traces: hm_0.txt, ts_0.txt, web_0.txt and proj_1.txt. proj_1.txt is read intensive, and the rest are write intensive. Tables I and Table II illustrate the results obtained, when LeaFTL used $\gamma = 0.0$, i.e., accurate mappings and $\gamma = 4.0$, respectively.

It can be observed that LeaFTL, on average reduces the size of the mapping table by 35.42%. For $\gamma = 0.0$, a 100% read accuracy is observed. When $\gamma = 4.0$ is used, the read accuracy decreases. For hm_0.txt trace, the drop in read accuracy is significant. In addition, the reduction in the size of the mapping table in the case of $\gamma = 0.0$ is minimal. For this reason, the authors evaluated performance improvements for LeaFTL considering $\gamma = 0$, and provided a separate study of sensitivity analysis on the impact of γ on the LeaFTL mapping scheme. For this term paper, improvements have been carried out considering $\gamma = 0.0$, which is the best performing case of LeaFTL.

IV. IMPROVEMENT STRATEGY

The reason that $\gamma = 4.0$ mapping table is not able to compact the table size by a significant amount is because the writes to SSDs are in the form of contiguous LPAs, which are approximated well enough using an accurate segment. However, for some write-cycles (The routine in which entries in write buffer are flushed to a free flash block), it was observed that the LPAs belonging to same frame had some discontinuities, normally, in a write routine, the LPAs belonging to a frame would be contiguous.

For example: consider LPAs of a frame: [1,2,3,4,5,6], these 6 LPAs are contiguous, but when there is a discontinuity like: [1,2,3,5,6], the accurate LeaFTL, when placing these LPAs learns 2 segments: [1,2,3] & [5,6], however, the approximate LeaFTL would still learn 1 segment [1,2,3,5,6] despite the discontinuity because of non-zero error margins. However, the size of discontinuity here, which is 1 (5-3 + 1), is rarely observed; the typical discontinuity size is large enough, so that even approximate LeaFTL learns 2 segments.

This is considered as a scope of improvement and uses a different rearrangement scheme that replaces a simple sort that LeaFTL does while placing pages into a block. The following figures provide a better picture of the cases discussed.

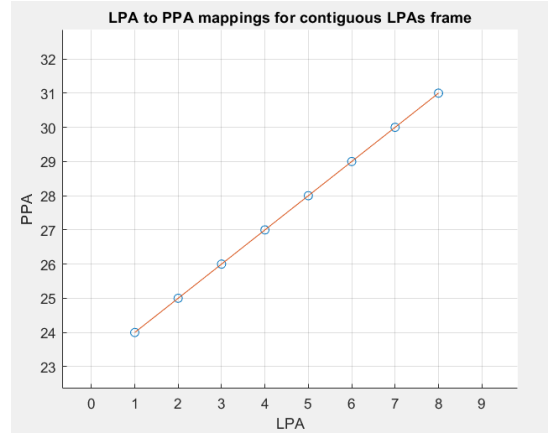


Fig. 4. For frames with contiguous LPAs written, a single segment is formed

LeaFTL sorts the writes in order of increasing LPAs, so even if a discontinuity is present, continuous physical pages are assigned to logical pages, leading to formation of multiple segments. If, for a frame containing discontinuous LPAs, some physical pages are skipped at discontinuities, then in principle, all the pages of that frame can be mapped using a single segment. The following figure describes this methodology.

The physical pages skipped in between can be assigned to other frames. This method works because, on average, only one or two frames may contain discontinuities during some of the write-routine, and for others, there would be no discontinuities observed in any of the frames. This makes it fairly easy to assign physical pages to logical pages during a write routine.

TABLE I
MAPPING TABLE SIZE REDUCTION IN LEAFTL $\gamma = 0.0$, WHEN COMPARED TO PAGE-LEVEL MAPPING SCHEME.
LEAFTL ACHIEVES 100 % ACCURACY IN PREDICTING THE LPA TO PPA MAPPINGS

Trace	Page Level mapping table size (MB)	LeaFTL mapping table size (MB)	Size Reduction (%)
hm_0.txt	126.31	82.72	34.50 %
ts_0.txt	126.06	81.17	35.61 %
web_0.txt	125.99	80.75	35.90 %
proj_1.txt	128.06	82.33	35.70 %

TABLE II
MAPPING TABLE SIZE REDUCTION IN LEAFTL $\gamma = 4.0$, WHEN COMPARED TO LEAFTL $\gamma = 0.0$.
THE ACCURACY IN PREDICTING THE LPA TO PPA MAPPINGS DROPS WITH INCREASED γ

Trace	LeaFTL ($\gamma = 0.0$) (MB)	LeaFTL ($\gamma = 4.0$) (MB)	Accuracy (%)
hm_0.txt	82.72	81.72	89.59 %
ts_0.txt	81.17	80.09	95.67%
web_0.txt	80.75	79.68	99.25 %
proj_1.txt	82.33	81.01	97.87 %

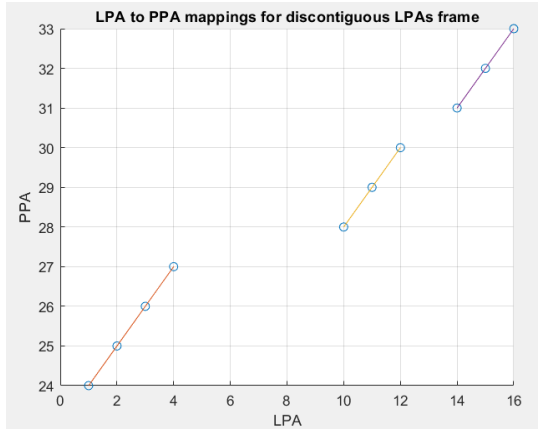


Fig. 5. For frames with discontinuous LPAs, multiple segments are formed. In this figure, because of 2 discontinuities, 3 segments are formed

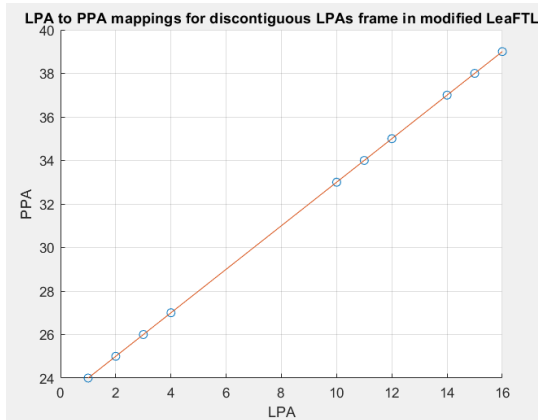


Fig. 6. For frames with discontinuous LPAs, a single segment can be formed, if physical pages are assigned keeping discontinuities in mind

A. Algorithmic Study

In the modified LeaFTL, let us call the frames having discontinuous LPAs to be written as discontinuous frames, and the their counterparts as continuous frames. The discontinuous frames are placed first, in the fashion described above. If the frame cannot be placed, it is skipped and placed along with other continuous frames. Placing discontinuous frames leads to fragmentation of the allocated block with multiple fragments of empty pages. For continuous frames and unplaced discontinuous frames, a contiguous space of page length equal to the number of LPAs is searched, and if found, the frame is placed; otherwise, it is placed in discontinuous fragments of empty pages in the block.

Note: It should be noted that this is just a rearrangement system. In the allocated block, the physical pages are written continuously, and not in fragments. Table III illustrates the results for modified LeaFTL.

TABLE III
MAPPING TABLE SIZE REDUCTION IN MODIFIED LEAFTL, WHEN COMPARED TO BASE LEAFTL

Trace	LeaFTL ($\gamma = 0.0$) (MB)	modified LeaFTL (MB)
hm_0.txt	80.047	80.048
ts_0.txt	80.047	80.048
web_0.txt	80.047	80.048
proj_1.txt	81.627	80.048

ACKNOWLEDGMENT

The authors thank Dr. T.V. Kalyan and Waqar Mir for continuous feedback received throughout the course of this project. We are also grateful to the Computer Science Department, IIT Ropar, for the Computing Resources provided making the simulation process faster and easier.

REFERENCES

- [1] Jinghan Sun, Shaobo Li, Yunxin Sun, Chao Sun, Dejan Vucinic, and Jian Huang. 2023. LeaFTL: A Learning-Based Flash Translation Layer for Solid-State Drives. In Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 442–456.